CS 4380 Project 3: Experimenting with Caching

Introduction

As you have learned, the concept of caching plays a critical role in the memory hierarchy (arguably it is the POINT of the memory hierarchy). In this project you will simulate and study the effects of adding various caches to the 4380.

Part 1: 4380 Emulator and Assembler Mark 2

In order to meet the requirements of this lab it will be necessary to extend the 4380 emulator and assemblers with support for the following additional instructions:

Move Instructions:

Value	Operator	Operand 1	Operand 2	Description
31	MOVI	RD	Imm	Move Imm value into RD*
8	LDA	RD	Address	Load address into RD**
22	ISTR	RS	RG	Store integer in RS at address in RG
23	ILDR	RD	RG	Load integer at address in RG into RD
24	ISTB	RS	RG	Store byte in RS at address in RG
25	ILDB	RD	RG	Load byte at address in RG into RD

* When represented in an assembly file the Imm value shall be represented as a numeric or character literal.

** When represented in an assembly file the Address shall be represented as a label.

JMP Instructions:

Value	Operator	Operand 1	Operand 2	Description
2	JMR	RS	Don't Care	Update PC to value in RS
3	BNZ	RS	Address	Update PC to Address if RS != 0*
4	BGT	RS	Address	Update PC to Address if RS > 0*
5	BLT	RS	Address	Update PC to Address if RS < 0*
6	BRZ	RS	Address	Update PC to Address if RS = 0*

* When represented in an assembly file the Address shall be represented as a label. Such labels MUST be associated with instructions.

Compare Instructions:

Value	Operator	Operand 1	Operand 2	Description
20	CMP	RD	RS	Set RD = 0 if RD == RS | Set RD = 1 if RD > RS | Set RD = -1 if RD < RS
32	CMPI	RD	Imm*	Set RD = 0 if RD == Imm | Set RD = 1 if RD > Imm | Set RD = -1 if RD < Imm

* When represented in an assembly file the Imm value shall be represented as a numeric or character literal.

Unless otherwise specified below - all requirements from projects 1 and 2 remain in force for the assembler and emulator.

Part 2: It's about time...

Caches are meant to increase the speed of a processor by reducing the number of clock cycles spent waiting on memory. To measure this speedup, we must first be able to measure the number of clock cycles spent on memory operations in general. To make this possible you must modify your emu4380 program to meet the following requirements:

Req1) The emulator must implement a global `unsigned int` variable called **mem_cycle_cntr**. This variable must be declared in emu4380.h and defined in emu4380.cpp, and shall be initialized to zero at program startup.

Req2) The emulator must contain the following memory access functions which must adhere to their associated definitions and descriptions:

- `unsigned char readByte(unsigned int address)`: Returns the unsigned char located at index `address` in **prog_mem**. Also increments the global **mem_cycle_cntr** by 8 when called.
- `unsigned int readWord(unsigned int address)`: Returns the unsigned int located at index `address` in **prog_mem**. Also increments the global **mem_cycle_cntr** by 8 when called.
- `void writeByte(unsigned int address, unsigned char byte)`: Places the value in byte at index `address` in the **prog_mem** array. Also increments the global **mem_cycle_cntr** by 8 when called.
- `void writeWord(unsigned int address, unsigned int word)` Also increments the global **mem_cycle_cntr** by 8 when called.

Req3) All programmatic memory accesses performed in the emulator (i.e. as the result of executing bytecode), including instruction fetches and data accesses, shall utilize the appropriate memory access functions described in Req2 above.

Req4) When a TRP #0 instruction is encountered - in addition to exiting with a return value of 0, the emulator shall first print the message: "Execution completed. Total memory cycles: NUM_CYCLES\n" - where NUM_CYCLES is the current value of **mem_cycle_cntr.**

Part 3: Adding our first cache

Now that your emulator is reporting the number of cycles spent accessing memory it is time to implement your first cache by extending your emulator to meet the following requirements:

Req5) The emulator shall accept an optional 2nd command line argument (in addition to the required input filename). This argument (if present) must be an unsigned integer value which shall indicate which of 3 available cache configurations is to be utilized by the emulator. The valid arguments are as follows:

- No argument : Default to no cache
- 0 : No cache
- 1 : Direct Mapped Cache
- 2: Fully Associative Cache

If an invalid argument is provided the emulator shall print the message "Invalid cache configuration. Aborting.\n", and exit with a return value of 2.

Req6) The emulator shall implement a direct mapped cache with 64 cache lines with block sizes of 16 bytes. The cache shall implement a write-back policy.

Req7) When the emulator is invoked with the optional command line argument indicating that a cache is to be used, the memory access functions described in REQ2 shall utilize the appropriate cache described in Req6 above or in Req? below to simulate cached memory accesses. When a memory access results in a hit within the cache the **mem_cycle_cntr** shall be incremented by 1 INSTEAD of 8. When a memory access results in a miss requiring a block be read from memory into the cache an additional 8 cycles shall be added to **mem_cycle_cntr**. If a memory access results in a miss that also causes an eviction; if the block currently present in the cache line must be written back to memory then an additional 8 cycles shall be added to **mem_cycle_cntr** (for a total of 17 cycles).

Part 4: Adding our second cache...

Part 5: Compare and contrast

Extra Credit: N-Way Set Associative (+10%)